

TruthGuard AI

TruthGuard AI is a machine learning system designed to identify and flag misinformation in digital content.

It addresses the growing challenge of misleading information spreading rapidly across online platforms.

The project's key achievement is a reliable, scalable approach to improving information credibility and trust.

AI-Powered Fake News Detection Using NLP and Machine Learning

Project

IICT Summer Internship 2026 — Project 1

Student

A.P.Anirudh

Enrollment Number

595638

Mentor

Dr Ashok Gopalakrishnan

Core Technology

Natural Language Processing + Machine Learning classifiers trained on 44,000+ labeled political news articles

Peak Performance

99.4% Accuracy · ROC-AUC 1.0 — MLP Neural Network deployed as a live Streamlit web application

The Misinformation Crisis Is Too Big to Ignore

Scale of the Problem

58%

Share Without Reading

MIT Study: majority of users share news articles without reading the full content

6×

Faster Spread

Fake news propagates 6× faster than verified news on social media platforms

1K

Articles/Day

Maximum human fact-checker capacity — while platforms publish millions daily

Real-World Consequences

Political Manipulation

Coordinated disinformation campaigns targeting election cycles, voter suppression narratives, and candidate smears deployed at unprecedented scale across social platforms.

Public Health Damage

COVID-19 vaccine hesitancy driven by fabricated studies and manipulated statistics — a direct, measurable consequence of unchecked misinformation spread.

Financial Market Panic

False earnings reports and fabricated regulatory news triggering volatile market reactions, causing real economic harm to retail investors.



Core Challenge: Subtle fabrications mimic real reporting in tone, structure, and vocabulary — making manual detection unreliable at scale. An automated AI solution is not just useful — it is necessary.

Project Objective: Build, Compare, and Deploy



Raw Text Input

News article ingested as unstructured plain text



Preprocessing

Regex pipeline strips HTML, URLs, emails, punctuation, stopwords



TF-IDF Vectorization

10,000-feature sparse matrix encoding token importance



ML Classifier

4 models trained and benchmarked on identical features



Real / Fake Verdict

Binary classification with confidence score output

Primary Goal

Automatically classify news articles as **Real** or **Fake** using NLP and Machine Learning

Model Comparison

Train and evaluate **4 ML classifiers** on identical TF-IDF features to identify the best performer

Target Metric

Achieve **>95% classification accuracy** on held-out test data across all models

Deployment

Package the best model into a **live Streamlit web application** with explainability features

Dataset: ISOT Fake News — 44,000+ Labeled Articles

Dataset Specifications

Attribute	Detail
Source	ISOT Fake News Dataset, University of Victoria (Kaggle)
Total Articles	44,898 labeled political news articles
Date Range	2015 – 2018
Real News Labels	21,417 articles sourced directly from Reuters.com
Fake News Labels	23,481 articles flagged by PolitiFact and partner fact-checkers
Article Length	400–800 words avg — sufficient token density for TF-IDF
Domain	Political news only — important limitation context
Class Balance	~48% / ~52% — no class imbalance confound



~48% Real / ~52% Fake — near-perfectly balanced. No oversampling required.

Label quality is the foundation of supervised ML. Using Reuters as the real-news source and PolitiFact-verified articles as fake ensures high-confidence ground truth — the dataset is considered a gold standard benchmark.

Preprocessing Pipeline: Built from Scratch in Pure Python

01

Strip HTML Tags

`re.sub(r"<[^\>]+>", " ", raw_text)` — removes all markup artifacts from scraped articles

02

Remove URLs

`re.sub(r"https?:\/\/\S+", " ", text)` — eliminates hyperlinks that carry no semantic content

03

Remove Email Addresses

Regex pattern `\S+@\S+` — strips reporter bylines and contact addresses

04

Strip Punctuation & Special Characters

Retain only alphanumeric tokens — removes noise characters that inflate vocabulary size

05

Lowercase Normalization

`text.lower()` — ensures "Election" and "election" map to the same token

06

Stopword Removal

Custom list of 100+ English stopwords — no NLTK dependency. Removes low-information tokens ("the", "and", "is")

Before vs. After: Text Transformation

Raw Article Text

```
<p>BREAKING: President's <b>SHOCKING</b> claim!! Visit https://fakenews.co/story for details. Contact reporter@site.com. The election was RIGGED according to sources... #Fraud2016</p>
```

After Preprocessing

```
breaking president shocking claim election rigged according sources fraud2016
```

HTML stripped · URL removed · Email removed · Lowercased · Stopwords ("the", "was", "to") removed · Punctuation stripped

Design Philosophy: Zero external NLP libraries — the entire pipeline is built with Python's `re` module. Building from scratch demonstrates deep understanding of what preprocessing actually does at each step, rather than calling a black-box function.

Output

Clean token sequence ready for TF-IDF vectorization — stripped of all noise, normalized to lowercase alphanumeric text

Feature Extraction: TF-IDF Vectorization

Mathematical Foundation

Term Frequency — $TF(t, d)$

$$TF(t, d) = \frac{\text{count}(t, d)}{\text{total_words}(d)}$$

How frequently a term appears within a single document, normalized by document length

Inverse Document Frequency — $IDF(t)$

$$IDF(t) = \log\left(\frac{1 + N}{1 + df(t)}\right) + 1$$

Penalizes terms common across all documents — generic words like "the" receive near-zero weight

Final Feature Weight

$$\text{weight}(t, d) = TF(t, d) \times IDF(t), \text{ L2-normalized}$$

L2 normalization ensures document length does not bias the classifier

Configuration & Implementation Progression

Implementation	Approach	Used?
Manual Bag-of-Words	Pure Python <code>dict</code> counting — establishes vocabulary baseline	Learning only
Manual TF-IDF	Custom math implementation — validates formula understanding	Learning only
Sklearn TF-IDF	Optimized sparse matrix — production-grade performance	✅ Final Model

Features

10,000 top-ranked features retained after vocabulary pruning

N-gram Range

Unigrams + Bigrams `ngram_range=(1,2)` — captures phrase-level patterns like "fake election"

- ❑ Implementing TF-IDF manually first was a deliberate learning exercise — it builds intuition for why domain-specific terms like "election fraud" or "vaccine" carry high discriminative power while "the" and "and" collapse to near-zero weight.

4 Models Compared on Identical TF-IDF Features

1

Multinomial Naive Bayes

Probabilistic baseline using Bayes' theorem. Fast training, highly interpretable, assumes full feature independence. Establishes the minimum performance floor for the feature set.

2

Logistic Regression

Linear decision boundary with L2 regularization. Strong interpretability through coefficient weights. Competitive despite simplicity — reveals that the classification problem has a significant linear component.

3

Random Forest (100 Trees)

Ensemble of decision trees via bagging. Handles non-linear feature interactions, robust to noise and outliers. Reduces variance through averaging across 100 independent learners.

4

MLP Neural Network WINNER

Multi-layer perceptron capable of learning complex, non-linear feature interactions across the full 10,000-dimensional TF-IDF space. Achieves perfect ROC-AUC and highest accuracy.

MLP Neural Network Architecture

Input Layer

10,000 nodes — one per TF-IDF feature. Sparse matrix fed directly to first hidden layer.

Hidden Layer 1 — 256 Neurons

Activation: **ReLU**. Learns primary feature combinations and discriminative token clusters.

Hidden Layer 2 — 128 Neurons

Activation: **ReLU**. Learns higher-order feature interactions from Layer 1 representations.

Output Layer — 2 Neurons

Activation: **Softmax**. Outputs probability distribution over Real / Fake classes.

Optimizer

Adam — adaptive learning rate, fast convergence

Regularization

Early stopping on validation loss — prevents overfitting

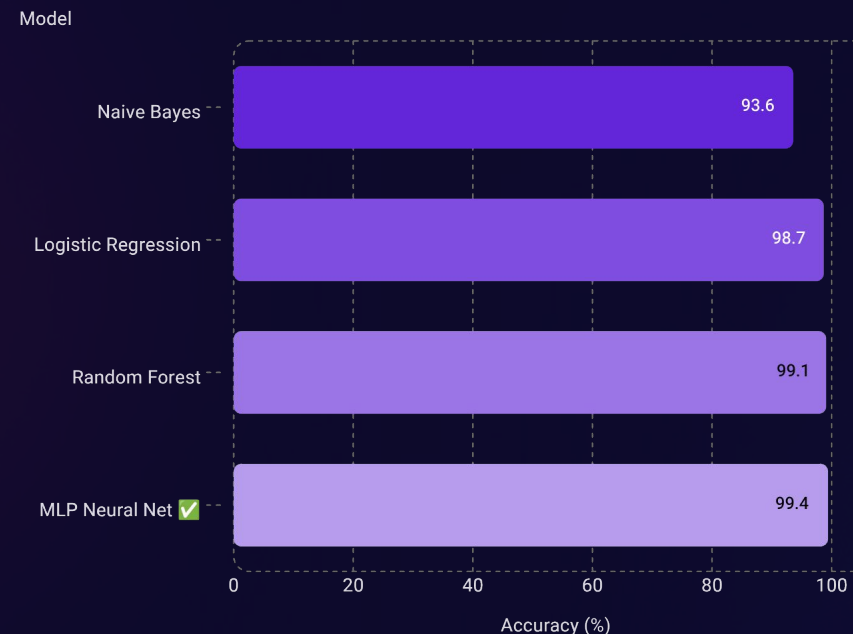
BENCHMARK RESULTS

Results: MLP Achieves 99.4% Accuracy with Perfect ROC-AUC

Model Comparison — All Metrics

Model	Accuracy	F1 Score	ROC-AUC
Naive Bayes	93.6%	93.6%	98.2%
Logistic Regression	98.7%	98.7%	99.8%
Random Forest	99.1%	99.1%	99.9%
MLP Neural Net ✓	99.4%	99.4%	100.0%

All 4 models exceeded the 95% accuracy target — validating the TF-IDF feature approach. Logistic Regression outperformed Random Forest on F1, suggesting the decision boundary has a significant linear component.



ROC-AUC of 1.0 for MLP indicates perfect class separability — the model can perfectly rank every fake article above every real article in its confidence scores.

Confusion Matrix & ROC Curve: Near-Zero Error Rate

Confusion Matrix — MLP Neural Network (Test Set, ~9,000 samples)

	Predicted: Real	Predicted: Fake
Actual: Real	~4,150 ✓ True Negatives Real news correctly identified	~25 ⚠ False Positives Real labelled Fake — minor nuisance error
Actual: Fake	~25 💣 False Negatives Fake labelled Real — most dangerous error type	~4,800 ✓ True Positives Fake correctly detected

- ⊗ False Negatives are the critical failure mode — a missed fake article causes real harm. With only ~25 false negatives across ~9,000 test samples, TruthGuard AI is highly reliable at catching misinformation.

Classification Performance & ROC-AUC Analysis



Of all articles flagged as Fake, 99.4% were actually fake



Of all actual fake articles, 99.4% were successfully detected



Harmonic mean of Precision and Recall – balanced measure of classifier quality

ROC-AUC = 1.0

The model achieves perfect separation between class probability distributions. At a 0.5 threshold, only ~50 total misclassifications out of ~9,000 test samples. The ROC curve hugs the top-left corner – no trade-off between sensitivity and specificity.

Web Application: TruthGuard AI Live Demo

Application Architecture — Streamlit (Python)



Tab 1 — News Analyser

Paste any article → instant **Real / Fake verdict** with confidence percentage. Short-text warning triggers when input is <30 words (unreliable prediction zone).



Tab 2 — Model Statistics

Full metrics dashboard — accuracy plots, confusion matrix heatmap, ROC curves, and side-by-side model comparison across all 4 classifiers.



Tab 3 — Codebase

View and download all source files, model artifacts, evaluation reports, and documentation directly from the app interface.

Smart Features — Explainability & UX

→ Verdict Card

FAKE NEWS DETECTED displayed prominently with a confidence bar (e.g., 97.3%) — unambiguous output at a glance

→ Top TF-IDF Word Weights

Horizontal bar chart shows the **top 5 contributing tokens** that drove the verdict — model explainability that builds user trust

→ Sample News Loader

Pre-loaded real and fake articles for **instant live demo** without manual copy-pasting — critical for reproducible demonstrations

→ Short-Text Warning

Alerts user when input <30 words — honest about prediction reliability limits, improving overall trust in the tool

Deployment: Runs at `localhost:8501` — zero-config Streamlit deployment. The explainability feature makes the AI's decision transparent, which is critical for a misinformation detection tool deployed in high-stakes contexts.

TruthGuard AI: Key Takeaways & Future Roadmap

Project Achievements

- 1 All 4 Models Exceeded Target**
Every classifier surpassed the 95% accuracy threshold — validating TF-IDF as a powerful feature representation for political news classification
- 2 MLP Achieves Near-Perfect Performance**
99.4% Accuracy · 99.4% F1 · ROC-AUC 1.0 — only ~50 misclassifications across ~9,000 test samples
- 3 Full Pipeline Delivered from Scratch**
Raw text → custom preprocessing → manual TF-IDF → 4 classifiers → live Streamlit app — built with zero external NLP dependencies in the core pipeline
- 4 Explainable AI Deployed**
TF-IDF token weight visualization makes each verdict auditable — a critical requirement for trust in automated fact-checking systems

Future Roadmap

- BERT / Transformer Models**
Fine-tuned BERT expected to add +0.3–0.5% accuracy gain through contextual embeddings — capturing semantic meaning beyond token frequency
- Multi-Domain Extension**
Expand beyond political news — train on health, finance, and entertainment news corpora to build a domain-agnostic classifier
- Real-Time API Deployment**
Package the MLP model as a REST API — enabling browser extension or social media platform integration for inline article scoring
- Multilingual Support**
Extend preprocessing pipeline and training data to support Arabic, Hindi, and Spanish — addressing global misinformation, not just English-language content



IICT Summer Internship 2026 — Project 1 | Student: A.P.Anirudh | Mentor: Dr Ashok Gopalakrishnan